

# Agriculture Support Triage Agent

An Agriculture Support Triage Agent is an AI assistant that reads farmers' messages, understands what they need, decides how urgent the issue is, extracts important information, and prepares a helpful reply automatically.

Think of it like a smart customer-support agent for farmers 🚜🌱.

## Objectives

- Accept farmer queries in natural language- Farmers can type their problems in normal everyday language and the system understands it automatically.
- Perform intelligent question understanding- The system interprets the actual meaning of the question and identifies the problem type.
- Extract crop, symptoms, and environmental context- The system automatically extracts important details such as crop, symptoms, and surrounding conditions.
- Evaluate severity of agricultural issues- The system determines whether the issue is normal, serious, or urgent.
- Generate actionable advisory responses- The system provides practical step-by-step solutions, not just explanations.
- Escalate high-risk cases when required- Critical cases are forwarded to an expert or authority for further assistance.

## System Flow

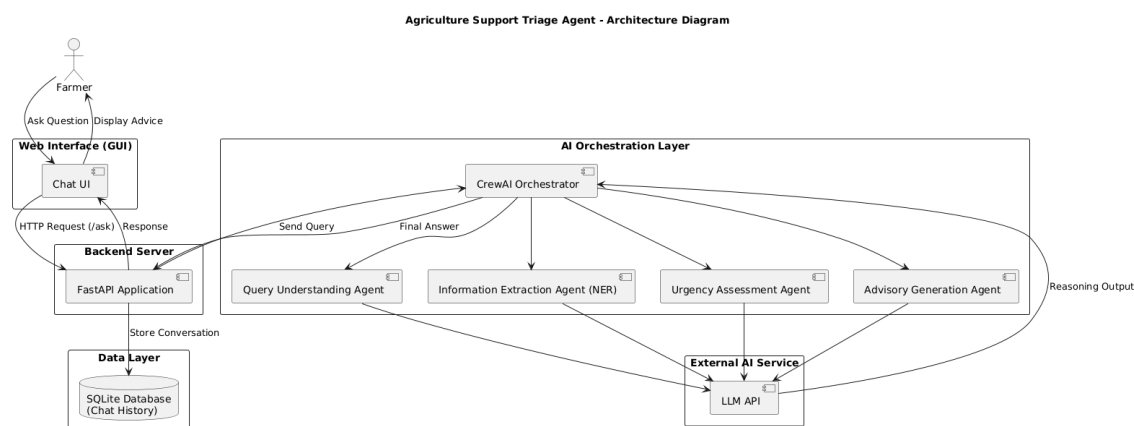
1. Farmer submit a query
2. CrewAI agents collaborate to analyze the query
3. Entities and context extracted
4. Urgency evaluated

5. Advisory generated
6. Response returned to farmer

## Tech Stack

- Python → Main programming language to build the system
- FastAPI → Backend API server (handles requests & responses fast)
- CrewAI → Manages AI agents and workflows
- LLM API → AI brain (understands questions & generates replies)
- SQLite → Lightweight database to store data locally
- Web GUI (Streamlit/Tkinter) → User interface where users interact with the app

## Architecture Diagram



### 1) Farmer Interaction (Input Stage)

1. The **Farmer** types a question in the **Chat UI**

Example: *“My wheat leaves are turning yellow. What should I do?”*

2. The GUI sends an **HTTP Request /ask** to the backend.

---

## 2) Backend Processing

3. The **FastAPI Application** receives the request

- Validates input
- Creates a session
- Sends the question to the AI system

→ The backend does NOT generate the answer  
→ It only acts as a bridge between UI and AI agents

---

## 3) CrewAI Orchestration (Brain of the System)

The request goes to the **CrewAI Orchestrator**

This is the most important component — it coordinates multiple AI agents.

Think of it like a **team leader assigning tasks to specialists**.

---

### AI Agents Collaboration

The orchestrator sends the same question to multiple agents, each having a different responsibility.

---

### Agent 1 — Query Understanding Agent

**Purpose:** Understand what the farmer is asking

It identifies:

- Is it disease?
- Fertilizer issue?
- Irrigation?
- Weather?

Example Output:

Category: Crop Disease

Uses → **LLM API**

---

## **Agent 2 — Information Extraction (NER) Agent**

**Purpose:** Extract important agricultural details

Finds:

- Crop: Wheat
- Symptom: Yellow leaves
- Stage: Growing phase (if mentioned)
- Location/time (if given)

Uses → **LLM API**

---

## **Agent 3 — Urgency Assessment Agent**

**Purpose:** Decide seriousness level

It checks risk level:

- Normal → general advice
- Important → treatment needed
- Critical → crop loss risk

Example Output:

Urgency: Important

Uses → **LLM API**

---

## **Agent 4 — Advisory Generation Agent**

**Purpose:** Produce the final farmer-friendly answer

Combines:

- Category
- Extracted entities
- Urgency

Generates:

“This appears to be nitrogen deficiency. Apply urea fertilizer and irrigate lightly.”

Uses → **LLM API**

---

#### **4) Returning the Final Answer**

All agent outputs go back to the **CrewAI Orchestrator**

The orchestrator merges them → **Final Answer**

---

#### **5) Storage**

FastAPI stores conversation in:

**SQLite Database (Chat History)**

Stored:

- Question
  - Answer
  - Timestamp
- 

#### **6) Response to Farmer**

7. Backend sends response to **Chat UI**

8. Chat UI displays advice to **Farmer**

#### **Future Enhancements**

- Image-based crop diagnosis
- Voice query support
- Weather-aware recommendations

